Presentation Script & Exam Q&A Prep
PART 1 — SPEAKING SCRIPT (English)
Slide 1 — Title
Slide 2 — Presentation
Slide 3 — Project at a Glance
Slide 4 — The Pipeline: Six Stages
Slide 5 — Two Independent Data Sources
Slide 6 — Software Architecture
Slide 7 — MySQL Relational Schema
Slide 8 — MongoDB Document Structure
Slide 9 — Data Cleaning: Two Issues Found & Fixed
Slide 10 — Integration & Enrichment
Slide 11 — Data Quality Assessment
Slide 12 — Results (1 of 2)
Slide 13 — Results (2 of 2)
Slide 14 — Limitations & Future Work
Slide 15 — Conclusions
PART 2 — LIKELY EXAM QUESTIONS & ANSWERS
A. Motivation & General Design Choices
B. Data Acquisition
C. Storage & Architecture
D. Data Cleaning
E. Integration & Enrichment
F. Data Quality Assessment
G. Queries & Results
H. Limitations, Validity & Critical Thinking
I. General Data Management Concepts (likely probed alongside the project)
J. SQL & Relational Query Specifics
K. MongoDB & NoSQL Specifics
L. Docker & Reproducibility
M. Data Modeling & Schema Design
N. General Database Theory
O. Testing, Validation & Software Engineering Practices
P. Ethics, Licensing & Data Governance
Q. Scalability & Performance
R. Alternative Approaches & Comparisons
S. Presentation & Communication
T. Sanity-Checking the Numbers

Presentation Script & Exam Q&A Prep

F1 2026: A Multi-Source Data Management Pipeline — Filippo Avon

---

PART 1 — SPEAKING SCRIPT (English)

General delivery notes before you start:

- Speak at a normal pace — don’t rush. Each slide below is roughly 30–50 seconds of natural speech.
- Don’t read the slide text aloud word-for-word; the script says the same things in a more natural spoken way.
- Pause briefly after stating a number (e.g. “twelve out of twelve races matched”) — let it land.

- If you forget a line, just describe what's on screen in your own words — you know this project better than the script does.

Slide 1 — Title

"Good morning/afternoon. My project is a multi-source data management pipeline built around the 2026 Formula 1 season. It combines a relational dataset and a semi-structured dataset, and takes them all the way from raw acquisition through to answering four research questions. Let me walk you through it."

Slide 2 — Presentation

"Before the technical detail, a quick word on the domain and the project. Formula 1 is the top tier of open-wheel motorsport — ten teams, about twenty drivers, roughly twenty-four Grands Prix a season worldwide. Every session generates dense timing, telemetry and strategy data, which makes it a genuinely rich domain for data management: high volume, multiple independent sources, and meaningful relationships between drivers, teams, races and circuits. In one paragraph, this project is a complete data management pipeline built around the 2026 season, combining a relational dataset and a semi-structured dataset from two independent public sources, covering the full lifecycle from acquisition to analysis — and built specifically to demonstrate measured, documented data quality at every step, not just a final answer."

Slide 3 — Project at a Glance

"Before going into detail, here's the project in numbers. I worked with two independent data sources, downloaded a total of forty-seven thousand seven hundred seventy-one raw files, and stored everything across two different database systems — one relational, one document-based. The integration between them reached a one hundred percent match rate for drivers and races, which I'll explain shortly. The whole pipeline answers four research questions, which you can see here — I'll come back to each of these with real results later."

Slide 4 — The Pipeline: Six Stages

"The project follows six stages: acquisition, storage, cleaning, integration and enrichment, data quality assessment, and finally analysis through four queries. Every stage is a separate, independent Python script — none of them depend on being run in a special environment, and all of them can be re-run safely without breaking anything. Both databases run inside Docker containers, which was a deliberate choice for reproducibility: anyone can recreate my exact environment with a single command."

Slide 5 — Two Independent Data Sources

"I used two sources that are structurally very different from each other. The first is a Kaggle dataset — a classic relational, Ergast-style set of CSV files covering every Formula 1 season from 1950 to 2026, which I downloaded with the kagglehub library and later filtered down to 2026 only. The second is TracingInsights, a GitHub repository that publishes session-level JSON data derived from FastF1 — things like raw telemetry, tyre compounds, and weather. I pulled all forty-seven thousand plus files from there through the GitHub REST API. Because these two sources have completely different shapes, I stored them in two completely different databases — which brings me to the architecture."

Slide 6 — Software Architecture

"Here's the full pipeline end to end. On the left, the Kaggle data flows through kagglehub into MySQL. On the right, the TracingInsights JSON flows through the GitHub API into MongoDB. Both go through their own cleaning step, then converge into an integration layer that builds a mapping between the two, followed by an enrichment step, a quality assessment, and finally the two query paths — SQL on one side, MongoDB aggregation on the other — that produce my four results. I chose SQLAlchemy and PyMySQL for the relational side, and PyMongo for the document side."

Slide 7 — MySQL Relational Schema

“This is the relational schema after cleaning. Everything revolves around the races table, which every fact table — results, qualifying, lap times, pit stops, standings — references through a raceId foreign key. The four lookup tables on the right — drivers, constructors, circuits, and status — were deliberately kept unfiltered, with their full historical data, because restricting them to 2026 only would have broken joins for absolutely no benefit.”

Slide 8 — MongoDB Document Structure

“On the MongoDB side, after cleaning I ended up with eight collections — down from an initial one hundred seventy-four, which I’ll explain on the next slide. The telemetry collection alone holds almost forty-six thousand documents, one per lap, with a raw speed array sampled at roughly 3.7 Hertz. Note the asterisks — those are the raceId and driverId fields that were added later, during enrichment, to link back to MySQL.”

Slide 9 — Data Cleaning: Two Issues Found & Fixed

“During cleaning I found and fixed two real problems. First, the Kaggle dataset spans the entire history of Formula 1, from 1950 to 2026 — I filtered it down to the 2026 season only, twenty-three races. Second, and more interesting: my first version of the MongoDB loader used each JSON filename as the collection name. Since TracingInsights stores one telemetry file per lap, that fragmented my telemetry data into about one hundred fifty separate collections instead of one. I rewrote the loader to route documents by the depth of their file path instead, which brought it down to eight clean, unified collections.”

Slide 10 — Integration & Enrichment

“MySQL and MongoDB don’t share a common key out of the box — MySQL uses numeric IDs, MongoDB uses human-readable codes like driver initials and Grand Prix names. So I built an automated mapping layer, and — importantly — I measured its accuracy rather than just assuming it worked. Twelve out of twelve races matched, eleven directly and one through a manual correction for a naming difference. Twenty-three out of twenty-three championship drivers matched. And zero duplicate records were found in either database. The key finding here is that every gap I found later has a documented explanation — it’s never a mystery.”

Slide 11 — Data Quality Assessment

“The brief specifically asks for data quality to be assessed both before and after integration, so I split this explicitly into two halves. Before integration, I checked each source completely independently: zero missing cells across all MySQL tables, all fifteen foreign key relationships one hundred percent valid with zero orphaned rows, and all seventeen completeness checks on MongoDB’s natural key fields also at one hundred percent. After integration, I measured the actual enrichment outcome — raceId and driverId coverage, and the match rates you saw on the previous slide. The key insight is that because both sources are independently clean, every gap I see after integration is explained by real differences in what the two sources cover — not by some pre-existing data problem.”

Slide 12 — Results (1 of 2)

“Now the results. Research question one: who had the fastest pit stop of the season? It was Franco Colapinto, twelve point zero nine two seconds, at the Dutch Grand Prix. Research question two: what was the highest speed recorded all season? Gabriel Bortoleto, three hundred sixty kilometers per hour, at Barcelona. I do flag this one, though — several of the top readings were recorded in the opening laps of a standing-start race, which is physically unlikely, so I treat this as a possible telemetry artifact rather than a confirmed record. I’ll say more about that in the limitations slide.”

Slide 13 — Results (2 of 2)

“Research question three: best race pace on medium tyres at Barcelona. That was Lewis Hamilton, at eighty-one point one one seven seconds per lap on average — though I flag that this is based on only twelve clean laps, compared to thirty-seven for Verstappen in second place, so I show that sample size transparently rather than hiding it. Research question four: best qualifying-to-race comeback. Fernando Alonso wins on both readings — the biggest single-race jump, eleven positions at Monaco, and the best season-long average, plus three point two five positions per race. The fact that the same driver tops both readings makes me fairly confident this is a real pattern, not a coincidence.”

Slide 14 — Limitations & Future Work

“Every project has limits, and I’d rather state mine clearly. The peak-speed anomaly I mentioned needs an outlier-detection step before it can be trusted fully. The tyre-pace comparison mixes very different sample sizes. The season was still ongoing when I acquired the data, so a few races only exist in MySQL so far. And one race name needed a manual mapping correction. Looking forward, I’d like to re-run this once the season finishes, and — this is the part I’m most excited about — extend it with a spatial analysis: TracingInsights has circuit corner coordinates, and MySQL has circuit latitude and longitude, so combining them with a spatial extension would let me analyze braking points and cornering speed relative to the track layout.”

Slide 15 — Conclusions

“To conclude: I built a complete, reproducible pipeline across two very different database systems, chosen specifically to match the shape of each data source. I measured integration accuracy explicitly rather than assuming it, reaching one hundred percent matches for drivers and completed races. Every quality gap I found has a documented cause. And the four research questions needed both sides of the project — two required plain SQL, two required MongoDB’s telemetry-level detail — which is really the whole point: combining the two sources gives you analysis that neither one could support alone. Thank you — I’m happy to take questions.”

PART 2 — LIKELY EXAM QUESTIONS & ANSWERS

Organized by topic. Read through all of these at least once — many databases/data-management courses probe both the specific project AND general concepts the project touches on.

A. Motivation & General Design Choices

Q1. Why did you choose two different DBMS instead of putting everything in one? Because the two sources have fundamentally different structures. Kaggle’s data is strongly relational — fixed columns, foreign-key relationships between races, drivers, results. Forcing MongoDB’s data into that same rigid shape would have been artificial and lossy, because TracingInsights’ JSON is heterogeneous and session-dependent (telemetry, weather, and lap times all have different structures). Using a relational store for the relational source and a document store for the document-shaped source lets each fit naturally, and it’s also a better demonstration of data management skills than picking one and forcing a fit.

Q2. Why Formula 1 2026 specifically? It offered two independent, actively maintained, real-world public data sources that are structurally different from each other (relational vs. semi-structured) but conceptually linkable through drivers and races — which is exactly what’s needed to demonstrate integration, enrichment, and quality assessment meaningfully.

Q3. Could you have used just one source and skipped integration entirely? Technically yes, but the assignment specifically requires integrating or enriching at least two datasets and measuring the associated errors — and pedagogically, the interesting part of this project (the key-mapping, the enrichment, the quality distinction) only exists because there are two sources to reconcile.

Q4. What would you have done differently if you designed this from scratch again? I’d build the driver/race key-mapping as a small reusable reference table from day one instead of a per-run CSV, and I’d add an outlier-detection step on raw telemetry before the peak-speed analysis, rather than discovering the anomaly after the fact. Both are already listed in my Future Work.

B. Data Acquisition

Q5. How did you actually download the data — manually or programmatically? Fully programmatically, with no manual steps. Kaggle data through the official `kagglehub` Python library, authenticated via a personal API token. TracingInsights data through the GitHub REST API — specifically the Git Trees API in recursive mode, with a fallback to the Contents API when the Trees API failed (which

happened intermittently given the repository's size).

Q6. Why did the GitHub API fail sometimes, and how did you handle it? The repository has tens of thousands of files, and GitHub's Git Trees API in recursive mode occasionally returned a five-hundred server error on a repository this large — not something in my control. I added automatic retries with backoff, and a slower-but-reliable fallback using the Contents API, which walks folders one at a time instead of requesting the whole tree at once.

Q7. How many files did you download, and were there rate limits to worry about? 47,771 JSON files from TracingInsights, downloaded in parallel with a thread pool for speed. For rate limits: GitHub allows only 60 unauthenticated requests per hour, which is far too few for a repository this size, so I used a personal access token to raise that to 5,000 requests per hour. Kaggle's download went through its own authenticated API with no comparable concern.

Q8. Why not just use the OpenF1 API instead of scraping GitHub? I actually started the project with OpenF1 as one of the sources, but changed direction: TracingInsights offered richer per-lap telemetry (tyre compound, raw speed samples) that better supported the specific research questions I ended up choosing, particularly the tyre-pace and peak-speed questions.

C. Storage & Architecture

Q9. Why MySQL specifically, and not PostgreSQL or SQLite? MySQL is a mature, widely used relational DBMS with strong Docker support, which fit the reproducibility goal well. SQLite would have been simpler but doesn't run as a proper client-server service, which matters less here but was a deliberate choice to mirror a more production-realistic setup. PostgreSQL would have worked equally well — my future-work notes actually mention PostGIS, a PostgreSQL spatial extension, as a possible upgrade path.

Q10. Why MongoDB and not another NoSQL database (e.g. Cassandra, CouchDB)? MongoDB's document model maps very naturally onto JSON, which is exactly the format TracingInsights publishes. It also has a mature, well-documented Python driver (PyMongo) and a powerful native aggregation framework, which I used directly for the peak-speed query instead of pulling all data into Python.

Q11. Why Docker for the databases? Reproducibility. The entire storage layer — both databases, their versions, their credentials — is defined in a single `docker-compose.yml` file. Anyone can recreate my exact environment with one command, without needing to install MySQL or MongoDB natively or worry about version mismatches.

Q12. Walk me through what happens if I run `docker compose up -d` on your project. It reads the compose file and starts two containers: a MySQL 8.0 instance seeded with an empty `f1_kaggle` database, and a MongoDB 7.0 instance with an `f1_tracinginsights` database, both exposed on their standard ports with the credentials defined in the file. At that point the databases are running but empty — the Python scripts then populate them.

Q13. What Python libraries did you use to talk to the databases, and why those? SQLAlchemy with the PyMySQL driver for MySQL — SQLAlchemy gives a clean, database-agnostic interface and integrates well with pandas' `read_sql`. PyMongo for MongoDB — it's the official driver and supports the native aggregation pipeline I used in Research Question 2.

D. Data Cleaning

Q14. What exactly was wrong with the data before cleaning? Two separate issues. In MySQL, the Kaggle dataset covers every season since 1950, but my project only needed 2026 — so without filtering, every query and every quality metric would have been diluted by 76 seasons of irrelevant data. In MongoDB, my first loading script used each JSON filename as its target collection name; since TracingInsights stores one file per lap of telemetry, that created roughly 150 near-duplicate collections instead of one.

Q15. How exactly did you filter MySQL to the 2026 season without breaking anything? I used `races.year == 2026` to get the set of valid `raceId` values for the season, then filtered every "fact" table (results, qualifying, lap_times, pit_stops, the two standings tables, sprint_results) to only rows whose `raceId` is in that set. Crucially, I did NOT filter the four lookup tables — drivers, constructors, circuits, status — because they have no season concept of their own and filtering them would only break joins (e.g. a driver's `driverId` still needs to resolve even if referenced from a 2026 race).

Q16. How did you fix the MongoDB collection fragmentation, technically? I rewrote the loading logic to route each JSON file to a collection based on the depth of its file path, rather than its filename. TracingInsights' repository structure has a predictable pattern: files at `<GrandPrix>/<Session>/<file>.json` are session-level data, while files at `<GrandPrix>/<Session>/<Driver>/<lap>_tel.json` are per-lap telemetry. Detecting that depth let me route all telemetry files into one unified `telemetry` collection with `driver_code` and `lap` as explicit fields, instead of one collection per lap number.

Q17. What was the `cor.json` issue, and why did you exclude it instead of fixing it? `cor.json` appeared in only 5 of the 69 sessions, while an equivalent, better-populated file, `corners.json`, existed in all 69. Rather than guess at reconciling two inconsistent, undocumented file types, I excluded the sparse one and documented the decision — that's a safer, more honest choice than silently merging possibly-inconsistent data.

Q18. How do you know your cleaning didn't silently break or lose data? I wrote a verification script (`check_databases_ready.py`) that checks every table's row count and every collection's document count against the expected values, and specifically checks the raceId enrichment coverage in MongoDB. I ran this after every reload, including after an unrelated Docker incident that wiped my containers, to confirm the databases were back to the expected state before continuing.

E. Integration & Enrichment

Q19. What exactly do you mean by "integration" versus "enrichment" here? Integration is figuring out the correspondence between the two sources — building the mapping tables that say "this MongoDB grand_prix value corresponds to this MySQL raceId", and "this MongoDB driver_code corresponds to this MySQL driverId". Enrichment is then actually applying that mapping: writing the resolved `raceId` and `driverId` values directly into the MongoDB documents, so that from that point on, every document carries its own link back to MySQL.

Q20. Why write the keys into MongoDB permanently instead of just joining at query time? Performance and simplicity for repeated use. Since I'd be running multiple different queries against the enriched data, computing the join once and persisting it avoids recomputing the same driver-code-to-driverId and grand-prix-to-raceId resolution on every single query. It also makes the data quality of the integration itself directly measurable and inspectable in the database, rather than hidden inside application logic.

Q21. How did you actually compute the mapping between driver codes and driver IDs? I compared the distinct `driver_code` values in MongoDB against MySQL's `drivers.code` column, restricted to drivers who appear at least once in the 2026 `results` table (to exclude irrelevant historical drivers). Matching codes were linked directly; codes appearing only in MongoDB were checked against the reserve/test driver list and confirmed to be drivers who only appeared in testing, never in an official race.

Q22. Why didn't all races match automatically, and how did you resolve it? One naming inconsistency: MongoDB calls it "Barcelona Grand Prix", MySQL calls it "Barcelona-Catalunya Grand Prix". I added a small manual mapping dictionary for this one case rather than trying to build a fuzzy-matching algorithm for a single known discrepancy — a targeted fix is more maintainable and auditable than a generic one when there's only one exception.

Q23. What does "update_many grouped by distinct key value" mean, and why did you do it that way? Instead of updating each of the 45,964 telemetry documents individually (which would mean 45,964 separate database round-trips), I grouped by the small number of distinct `grand_prix` and `driver_code` values — at most a few dozen — and issued one `update_many` operation per distinct value, each of which updates every matching document in a single database call. This is dramatically more efficient at scale.

Q24. How did you measure the "error rate" of your integration, as the assignment requires? Two explicit match-rate metrics: races matched (12 out of 12 completed races, with 3 pre-season testing sessions correctly excluded as out of scope) and drivers matched (23 out of 23 championship drivers). Every non-match was individually investigated and attributed to a specific, documented cause — not left as an unexplained gap.

F. Data Quality Assessment

Q25. Why does the assignment ask for quality before AND after integration, and how did you

interpret that? Because a percentage measured only after integration is ambiguous — you can't tell whether a gap comes from bad source data or from a real difference in what the sources cover. I first assessed each source's quality completely independently, immediately after cleaning: MySQL's completeness and referential integrity, MongoDB's completeness on its own natural-key fields. Only after confirming both sources were internally clean did I measure the post-integration outcome — enrichment coverage and match rates — which lets me say with confidence that any post-integration gap reflects real coverage differences, not pre-existing defects.

Q26. What is referential integrity, and how did you check it? Referential integrity means every foreign key value in a "child" table actually exists in the referenced "parent" table — e.g., every `raceId` in the `results` table should exist in the `races` table. I checked this explicitly with a left join between each child table and its parent, counting rows where the parent side comes back null (an "orphaned" row). I ran this for all fifteen foreign-key relationships in my schema, and all fifteen came back at 100% valid, zero orphans.

Q27. What's the difference between completeness, uniqueness, and consistency in your assessment? Completeness measures whether values are actually present — no missing cells in MySQL, and all the natural-key fields present in MongoDB. Uniqueness measures whether records that should be distinct actually are — no duplicate `results` rows for the same driver and race, no duplicate telemetry documents for the same driver, session, and lap. Consistency, in my case, is about cross-source agreement — the match rates between MySQL and MongoDB identifiers.

Q28. If everything came back at 100%, doesn't that mean your quality assessment wasn't very useful? On the contrary — proving that both sources are clean is itself a meaningful and non-trivial result, because it's what allows me to make a much stronger claim afterward: that every post-integration gap is fully attributable to genuine coverage differences (testing sessions, reserve drivers) rather than data quality problems. A quality assessment that finds nothing wrong is only "not useful" if you assumed the answer in advance — here it was a real check, and it changed how confidently I could interpret the enrichment results.

Q29. Are there any data quality issues you found that you didn't fully resolve? Yes — the peak-speed anomaly in Research Question 2. Several top readings occur in the opening laps of a standing-start race, which is physically implausible. I didn't have time to build a full outlier-detection pipeline, so I flagged it explicitly in both the report and the presentation rather than either hiding it or quietly excluding it without disclosure.

G. Queries & Results

Q30. Why did two questions need SQL and two need MongoDB — couldn't you answer all four from one database? No, not without losing accuracy or information. RQ1 (pit stops) and RQ4 (qualifying-to-race positions) only need data that lives in MySQL — pit stop durations, grid and finishing positions. RQ2 (peak speed) and RQ3 (tyre-compound pace) need data that exists only in MongoDB — raw telemetry speed samples and tyre compound per lap — neither of which Kaggle's relational dataset contains at all.

Q31. Real F1 pit stops are famous for being under 2 seconds — why is your fastest result 12 seconds? Is that a data error? No, it's a difference in what is actually being measured, and I disclose this explicitly as a limitation. The sub-2-second figures widely reported in the media measure only the stationary time — the moment the car is jacked up and the tyres are changed. The Kaggle/Ergast `pit_stops.duration` field I used measures the full pit-lane time: from the moment the car crosses the pit-lane entry line to the moment it crosses the exit line, which includes deceleration, the drive to the box, the stop itself, and acceleration back onto the track. That is a real and meaningful metric — it's just a different one, and the two aren't directly comparable. I added this distinction to my Limitations precisely so the result isn't misread.

Q32. For RQ2, why use MongoDB's aggregation framework instead of just pulling the data into Python with pandas? Efficiency and demonstrating proper use of the database engine. The `$max` aggregation operator can compute the maximum value of the `speed` array inside every document, server-side, in a single pipeline call — rather than transferring the raw high-frequency telemetry for all 45,964 laps over the network and computing the maximum in Python. It's both faster and a better demonstration of MongoDB's native capabilities.

Q33. Explain exactly what the `$max` operator did in your RQ2 query. Within a single `$project` stage, `{"$max": "$tel.speed"}` takes the `speed` array nested inside each document's `tel` field and returns its single largest value as a new field, `speed_max_kmh`. I did this for every document, then sorted descending and took the top result — all inside the database, before any data left MongoDB.

Q34. In RQ3, why did you exclude pit-in/pit-out laps and inaccurate laps before computing the average? Because including them would bias the pace comparison unfairly. A pit-in or pit-out lap includes time spent slowing down for or accelerating out of the pit lane, which has nothing to do with genuine race pace on that tyre compound. Similarly, FastF1's `iacc` (is-accurate) flag marks laps whose timing data failed a basic sanity check — including those risks contaminating the average with bad measurements rather than real driving differences.

Q35. You mention TracingInsights uses the string "None" instead of a real null — how did that affect your code, and how did you catch it? My first version of the RQ3 filter checked for real `null` / `NaN` values using pandas' `.isna()`, and it returned zero matching laps for every driver — clearly wrong. I added diagnostic print statements after each filtering step, which showed the count dropping to zero specifically after the pit-in/pit-out filter, which pointed me to the actual field values — and I found they were literally the text string `"None"`, not a null. I fixed it with a helper function that treats both the real null and the string `"None"` as "no value".

Q36. For RQ4, why report both a single-race record AND a season-long average instead of just one? Because they answer subtly different questions and can point to different drivers. A single-race record captures a spectacular one-off recovery; a season-long average captures consistent race-day performance. I report both for a fuller picture — and in this case, they reinforce each other, since Fernando Alonso leads on both, which increases my confidence that it reflects a genuine skill or strategy pattern rather than a lucky outlier.

Q37. How exactly is "positions gained" computed in RQ4? `positions_gained = qualifying_position - race_finishing_position`, joining the `qualifying` and `results` tables on race and driver. A positive value means the driver finished higher than they qualified (a comeback); negative means they lost positions during the race.

Q38. Why is Hamilton's RQ3 result based on only 12 laps while Verstappen's is based on 37 — isn't that a problem? It reflects real tyre-strategy differences during the race — some drivers spent much more of the race on medium compound than others, which isn't something I can or should normalize away, since it's a genuine strategic choice, not a data error. I addressed it by reporting the clean-lap count transparently alongside every average, so the comparison's reliability is visible rather than hidden — a reader can decide how much weight to give a 12-lap average versus a 37-lap one.

H. Limitations, Validity & Critical Thinking

Q39. How confident are you that the RQ2 result (360 km/h) is real? Not fully confident, and I say so explicitly in the report and presentation. The value itself is plausible for an F1 car on a fast straight, but the fact that several of the top ten readings cluster in the first few laps of a standing-start race — when cars are still bunched up on the formation lap and can't realistically be at full speed — is a red flag. My working theory is a GPS or telemetry interpolation artifact in that specific session, not a genuine on-track record, but I did not have time to build a rigorous outlier-detection step to confirm it, so I flagged it as a limitation instead of either suppressing it or asserting it as fact.

Q40. Since the 2026 season wasn't finished when you did this project, how does that affect your results? It mainly affects RQ4's season-long average, which reflects only the races completed at acquisition time (12 races), not a final championship picture — Alonso's average could shift by season's end. It also explains part of the race-matching gap: races not yet run exist only in MySQL's calendar and correctly have no MongoDB counterpart yet, which is an expected, time-bound gap rather than an integration failure.

Q41. If you had to defend the single biggest weakness of this project, what would it be? Probably the RQ2 peak-speed result — it's the one finding I'm reporting without being able to fully verify it, and I think it's important to be upfront that "the query correctly executed" and "the result is physically meaningful" are two different claims. I chose to disclose that gap rather than paper over it, which I think is the right call methodologically, but it does mean that specific result needs a caveat every time it's cited.

Q42. Why didn't you pursue the spatial integration angle the assignment specifically encouraged? Time and scope. I judged that doing four well-executed, clearly validated research questions with rigorous quality assessment was a stronger deliverable than five questions where the spatial one was rushed. I did lay the groundwork for it, though — TracingInsights' `corners` collection already has corner coordinates enriched with `raceId`, ready to be joined with MySQL's `circuits.lat` / `lng` — and I describe exactly how I'd do it in Future Work.

I. General Data Management Concepts (likely probed alongside the project)

Q43. What's the general trade-off between a relational and a document database, in your own words? Relational databases enforce a fixed schema and strong consistency guarantees through foreign keys and constraints, which is ideal when your data genuinely has a stable, tabular structure with clear relationships — like race results. Document databases trade some of that structural rigidity for flexibility: each document can have a different shape, which is ideal when your data is naturally nested or varies from record to record — like telemetry sessions with different available fields per session type. Neither is universally "better"; the right choice depends on matching the model to the actual shape of the data, which is exactly the reasoning I used here.

Q44. What is a foreign key, and why does it matter for data quality? A foreign key is a column (or set of columns) in one table that references the primary key of another table, enforcing that a value can only exist if the row it points to actually exists. It matters for data quality because it's the mechanism that prevents "orphaned" records — like a race result pointing to a driver that doesn't exist in the drivers table — which is exactly what my referential-integrity check in the quality assessment was verifying.

Q45. What does "idempotent" mean, and why did you make your scripts idempotent? An idempotent operation produces the same end result no matter how many times you run it. I made every script in my pipeline idempotent — mostly through `if_exists="replace"` when writing to MySQL and dropping/recreating collections in MongoDB — specifically so that if something went wrong partway through (which did happen once, when Docker reset my containers), I could simply re-run the affected scripts from scratch without any risk of duplicating data or ending up in an inconsistent state.

Q46. What's the difference between data cleaning and data integration, conceptually? Cleaning improves the quality of a single source in isolation — removing irrelevant rows, fixing structural problems, handling missing or duplicate values — without reference to any other dataset. Integration is specifically about reconciling two or more sources that describe overlapping real-world entities but identify them differently, which requires building a correspondence between them — in my case, the driver-code-to-driverId and grand-prix-to-raceId mappings.

Q47. Why is measuring an "error rate" important in a data integration project, rather than just performing the integration? Because integration without measurement is just a guess dressed up as a result. Any two datasets can be forced together with enough assumptions, but unless you can quantify how many records matched correctly, how many didn't, and why, you have no way of knowing whether your combined dataset is trustworthy enough to base further analysis on. That's why the assignment explicitly requires it, and why I built the match-rate measurement as a first-class part of the pipeline rather than an afterthought.

J. SQL & Relational Query Specifics

Q48. What SQL join types did you use across your queries, and why? Every join I used was an INNER JOIN, because in all four cases I only wanted rows that have a match on both sides — a pit stop only makes sense paired with the driver and race it belongs to, and a qualifying record only makes sense paired with an existing race result. I did use LEFT JOINs elsewhere, though — specifically in the referential-integrity checks in my data quality assessment, where a LEFT JOIN that comes back NULL on the parent side is exactly how you detect an orphaned foreign key.

Q49. In RQ1, why order by milliseconds rather than the duration string field? The `duration` field in the Kaggle/Ergast schema is stored as a string (e.g. "12.092"), and sorting strings lexicographically gives wrong results once numbers have different digit counts (e.g. "9.5" would sort after "12.0" as text). The `milliseconds` field is a genuine integer, so ordering by it gives a correct numeric sort; I only used `duration` for display formatting in the final table.

Q50. How would you adapt RQ4 to compute average positions gained per constructor instead of per driver? I'd join `qualifying` and `results` exactly as before, but group by `constructorId` from `results` instead of `driverId`, and average `positions_gained` per constructor. It's essentially a one-line change to the GROUP BY clause, which is one of the reasons I designed the query logic to be simple and reusable rather than a one-off calculation.

Q51. What indexes would you add to your MySQL schema to speed up the four queries, and why?

`raceId` and `driverId` already have supporting indexes since they're primary/foreign keys in this schema. If I were tuning further, I'd add an index on `pit_stops(milliseconds)` to speed up RQ1's ORDER BY, and composite indexes on `qualifying(raceId, driverId)` and `results(raceId, driverId)` to speed up the join in RQ4. At this data volume — a few hundred to a few thousand rows per table — it wasn't a real performance concern, but it would matter at production scale.

Q52. Why did you use SQLAlchemy's `read_sql` with pandas rather than raw `cursor.execute()`? `pandas.read_sql` returns a DataFrame directly, which lets me reuse the same downstream logic — sorting, formatting, exporting to CSV, feeding into matplotlib — that I already use everywhere else in the project. Writing raw cursor code would mean manually mapping rows to columns before I could do anything useful with the result, for no real benefit at this data volume.

Q53. What would happen to your RQ1 query if a driver had a NULL pit stop duration? Because I used an INNER JOIN and ordered by `milliseconds` ascending, a NULL value would sort first in MySQL under default ascending order and could wrongly appear as the "fastest" stop. I didn't hit this in practice — the completeness check in my data quality assessment already confirmed 0% missing cells in `pit_stops` — but if it had occurred, the fix would be an explicit `WHERE milliseconds IS NOT NULL`.

K. MongoDB & NoSQL Specifics

Q54. What's the difference between `$match` and `$project` in a MongoDB aggregation pipeline, and where did you use each? `$match` filters which documents continue through the pipeline, like a WHERE clause; `$project` reshapes each surviving document, selecting or computing new fields, like a SELECT clause. In RQ2 I used `$match` first to discard telemetry documents without a `speed` array, then `$project` to compute a new `speed_max_kmh` field from that array using `$max`.

Q55. Why does the telemetry data use an embedded array structure (`tel.speed`) instead of one document per sample? That's the structure TracingInsights already publishes — one JSON file, and therefore one document, per lap, with all ~3.7 Hz samples for that lap embedded as parallel arrays. Designing telemetry storage from scratch, one document per sample would give finer query flexibility but at a much higher document count and overhead; the embedded-array approach is a reasonable trade-off for data that's almost always analyzed one full lap at a time anyway.

Q56. What are the trade-offs of the columnar (array-of-arrays) structure you found in `laptimes` and `weather`? It's compact and fast to load in one piece, since a whole session's data comes back in a single document read. The downside is that it doesn't take advantage of MongoDB's native per-document querying — you can't easily ask the database "give me just lap 5" without pulling the whole document and filtering client-side, which is exactly the manual unwinding I had to do for RQ3.

Q57. How would you index the `telemetry` collection to speed up your RQ2 aggregation? Since the aggregation has to scan every document to compute `$max` on `tel.speed`, a normal index doesn't help that scan directly. I would, however, add a compound index on `{grand_prix: 1, session: 1}` to speed up any query that first filters to a specific race weekend before aggregating, which is a very likely follow-up query pattern.

Q58. What does "schema-less" really mean for MongoDB, and did your data actually take advantage of that flexibility? It means MongoDB doesn't enforce a fixed set of fields or types across documents in a collection, the way a relational table enforces columns. My data did take advantage of it in a real, if limited, way: `laptimes` documents have different fields depending on session type — qualifying sessions carry a `qs` field for Q1/Q2/Q3 that race sessions simply don't have — which a rigid relational schema would have forced me to handle with nullable columns or a separate table.

L. Docker & Reproducibility

Q59. What happens to your data if you run `docker compose down` versus `docker compose down -v`? `docker compose down` stops and removes the containers but keeps the named volumes, so MySQL and MongoDB's data survives and reappears the next time you run `docker compose up -d`. `docker compose down -v` additionally deletes the volumes, wiping both databases completely — this is actually close to what happened accidentally partway through my project when Docker had a problem, which is exactly why I built the verification script in the first place.

Q60. How do the credentials in your `docker-compose.yml` relate to security best practice — would you do anything differently for production? My credentials are simple and hard-coded directly in the compose file, which is fine for a local, throwaway development environment but would be a real problem in production. For a production system I'd use Docker secrets or environment variables injected at deploy time, never commit credentials to version control, and use less privileged database users than `root`.

Q61. If someone had a different OS (say Mac instead of Windows), would your pipeline still work? What would need to change? The Docker and database layer would work identically, since containers are OS-independent. The only things that would need to change are the hard-coded Windows-style absolute paths (like `C:\Users\filip\Desktop\DataMAN`) in my scripts, which would need to become Mac/Linux-style paths — a one-line edit per script, not a structural change.

M. Data Modeling & Schema Design

Q62. Why did you not normalize the MongoDB documents into more granular collections, e.g. one document per telemetry sample? It would multiply the document count by roughly the number of samples per lap — telemetry at 3.7 Hz over a ~90-second lap is around 330 samples, so 45,964 lap-documents would become somewhere around 15 million sample-documents. That buys finer-grained queryability I didn't need, at a real cost in storage and in the complexity of every downstream aggregation, so I kept the source's own one-document-per-lap granularity.

Q63. Is your MySQL schema in third normal form? Explain with an example from your own tables. Largely yes — `results` stores `constructorId` as a foreign key rather than repeating the constructor's name and nationality on every row, which is exactly what 3NF requires: non-key attributes should depend only on the key, not on other non-key attributes. The one place I'd flag as a deliberate exception is `results.fastestLapSpeed`, which is derived rather than a pure fact — but that's inherited from the Ergast source schema, not a choice I made.

Q64. If you had to model this domain from scratch without pre-existing datasets, what would your ideal schema look like? I would keep the relational core very close to what Kaggle/Ergast already provides, since races-drivers-constructors-results is a genuinely well-normalized model for this domain. For the telemetry side I'd likely still choose a document store, and I'd still choose one document per lap per driver, for the same storage/query trade-off discussed in Q62.

N. General Database Theory

Q65. What does ACID stand for, and does MySQL guarantee it in your setup? Does MongoDB? Atomicity, Consistency, Isolation, Durability. MySQL with the InnoDB engine (the default, which I used) fully supports ACID transactions. MongoDB guarantees atomicity at the single-document level always, and has supported full multi-document ACID transactions since version 4.0 — though I didn't need multi-document transactions anywhere in this project, since my writes (like the enrichment step) are idempotent and safely re-runnable even without transactional guarantees.

Q66. What is the CAP theorem, and how does it relate to your choice of MongoDB? CAP theorem says a distributed database can't simultaneously guarantee Consistency, Availability, and Partition tolerance — you have to trade off at least one. It isn't really a live concern in my project because I ran a single MongoDB instance, not a distributed cluster, so there's no network partition to reason about; CAP becomes directly relevant if this project were scaled out to a MongoDB replica set or sharded cluster.

Q67. What's the difference between a clustered and non-clustered index, and does it matter here? A clustered index determines the physical storage order of a table's rows (there can only be one, usually the primary key); a non-clustered index is a separate structure pointing back to the rows. In InnoDB, my primary keys (`raceId`, `resultId`, etc.) are clustered indexes by default, which matters most for range scans and joins on those keys — exactly what my queries do — but at my data volume it wasn't something I needed to tune manually.

Q68. What is a transaction, and did you need one anywhere in your pipeline? A transaction is a group of database operations that either all succeed or all fail together, preserving consistency even if something goes wrong partway through. I didn't explicitly wrap anything in a transaction, because every write in my pipeline is either a single bulk load (safe to just re-run entirely if it fails) or an idempotent `update_many` grouped by key — the idempotency substitutes for needing transactional rollback in this specific project.

Q69. What's the difference between OLTP and OLAP, and which does your project resemble more? OLTP (online transaction processing) systems are optimized for many small, fast reads and writes — think a live booking system. OLAP (online analytical processing) systems are optimized for complex aggregate queries over large volumes of historical data — think a reporting dashboard. My project is much closer to OLAP: I'm not processing live transactions, I'm running aggregate analytical queries — averages, maximums, rankings — over a fixed historical dataset.

O. Testing, Validation & Software Engineering Practices

Q70. How did you test that your scripts worked correctly before trusting their output? Mainly through the verification and data-quality scripts built specifically for this purpose: `check_databases_ready.py` confirms row/document counts match expectations after every load, and `data_quality_report.py` checks completeness, referential integrity, and duplicates. For trickier logic, like the RQ3 filtering, I used diagnostic print statements after each filtering step, which is exactly how I caught the "None"-string bug before trusting the final numbers.

Q71. Did you use version control (e.g. Git) for this project? *[Personalize this with your real workflow: if you used Git, briefly describe your commit habits — e.g. "yes, I committed after each pipeline stage was working, which let me pinpoint exactly when the MongoDB fragmentation bug was introduced." If you worked directly in the notebook without Git, say so plainly and note that for a larger or team project you would add it from day one for exactly the kind of debugging Q6/Q34 describe.]*

Q72. What would a unit test for your enrichment script look like? I'd construct a small in-memory (or mocked) MongoDB collection with a handful of documents with known `grand_prix` and `driver_code` values, run the enrichment function against a known mapping table, and assert that the resulting `raceId` / `driverId` fields match expectations — including at least one document whose key is deliberately absent from the mapping, to confirm it's correctly left as `None` rather than crashing or silently mismatching.

P. Ethics, Licensing & Data Governance

Q73. Are you allowed to use the Kaggle dataset and the TracingInsights repository for this purpose? What are the licensing terms? Both sources are publicly available: the Kaggle dataset is published by its author on Kaggle's platform, and I used it strictly for non-commercial academic coursework, the standard permitted use case for this kind of dataset. TracingInsights is an openly accessible public GitHub repository. I didn't do a formal legal license review beyond confirming both are intended for public/academic use, which would be the natural next step before any commercial application of this work.

Q74. Does your project raise any privacy concerns, given it involves real people's names and performance data? The data involves public figures — professional F1 drivers — acting in their public professional capacity, and race results and telemetry are already broadcast and published by the sport itself, so this is quite different from handling private personal data. If I were extending this approach to, say, amateur racing data involving private individuals, I would need to think much more carefully about consent and anonymization, but that isn't a concern for the data used here.

Q. Scalability & Performance

Q75. If TracingInsights had 10x more data, what would break first in your pipeline? Almost certainly the GitHub Trees API call during acquisition — it already occasionally fails with a 500 error at the current size, and a 10x larger repository would make that worse and push more of the acquisition onto the slower Contents-API fallback. After that, the MongoDB enrichment step would take proportionally longer, though it would scale much better than a naive per-document approach would, since it's already grouped by distinct key rather than per-document.

Q76. How long did the full pipeline take to run end-to-end, and what was the slowest step? *[Personalize with your real timing if you tracked it. Qualitatively: acquisition — downloading 47,771 files subject to GitHub's rate limits and occasional retries — was almost certainly the slowest stage; cleaning, integration and the four queries each completed in a comparatively short time once the databases were populated.]*

R. Alternative Approaches & Comparisons

Q77. Could you have done this whole project with just pandas and CSV files, without any DBMS at all? Technically the final query logic could be replicated with pandas alone, but the project brief explicitly requires acquisition, storage, integration and quality assessment using real DBMS, and doing so also demonstrates skills — schema design, referential integrity, indexing, aggregation pipelines — that a flat-file approach simply doesn't exercise. A pandas-only approach would also scale far less gracefully if the telemetry volume grew significantly.

Q78. What would change if you used PostgreSQL with a JSONB column instead of MongoDB? I could store the TracingInsights JSON documents as JSONB columns in PostgreSQL and query into them with PostgreSQL's JSON operators, keeping everything in a single DBMS. The trade-off is that PostgreSQL's JSON querying syntax is less naturally expressive for deeply nested, array-heavy documents than MongoDB's native aggregation framework, and I'd lose the clear architectural story — the right tool for each source's shape — that the two-DBMS design demonstrates.

Q79. Why not use a data warehouse or an orchestration tool like Apache Airflow for this pipeline? Both would be reasonable choices at larger scale or in a team/production setting, but they'd be disproportionate for a project of this size — Airflow's value is in scheduling, retrying and monitoring many interdependent pipelines over time, whereas mine is a linear, one-shot sequence of scripts run interactively. I kept the orchestration as simple, independent Python scripts specifically so the logic stays transparent and easy to inspect for a course project.

S. Presentation & Communication

Q80. Why did you structure your presentation with a "Presentation" overview slide before diving into technical details? Not every audience can be assumed to know the domain, so a short abstract of what Formula 1 actually involves — teams, drivers, race weekends, the kind of data each session generates — makes the rest of the technical content land better, without assuming prior knowledge that not everyone in the room necessarily has.

Q81. If you only had 5 minutes to present this, what would you cut? I'd cut the two schema-diagram slides and compress the two results slides into one, keeping the project-at-a-glance numbers, the pipeline overview, the integration/quality highlights, and one or two headline results — that keeps the strongest, most quantifiable claims (the 100% match rates, the before/after quality distinction) while dropping the more detailed structural walkthroughs, which I could still cover if asked directly.

T. Sanity-Checking the Numbers

Q82. Why does the MySQL races table have 23 rows but only 12 races matched to MongoDB? The 23 rows are every race on the 2026 calendar as Kaggle already lists it, including races that hadn't been run yet at the time I acquired the data. Only 12 races had actually taken place and therefore had corresponding session data published in TracingInsights, so only those 12 could be matched — the other 11 correctly remain unmatched until those races actually happen and TracingInsights publishes their data.

Q83. Why is `telemetry` 45,964 documents while `laptimes` is only 1,464? `telemetry` has one document per lap per driver — the full per-lap continuous data. `laptimes` has one document per driver per session, with all of that driver's laps for the whole session embedded as arrays inside a single document. The same underlying laps are represented at very different granularities in the two collections, which is why the document counts differ by roughly a factor of 30.

Q84. Why did you use matplotlib instead of a more modern library like Plotly or Seaborn for your charts? matplotlib is fully sufficient for the static, publication-quality bar charts I needed for the report and slides, and gives precise control over every visual element — colors matching my F1-inspired palette, exact bar highlighting — with no extra runtime dependency for the reader, since the outputs are just PNG images embedded in a document. Plotly's main advantage, interactivity, wouldn't even be usable inside a static PDF or PPTX.

Q85. How did you decide which of the many possible research questions to actually pursue? I deliberately chose four that, between them, would require touching both databases individually and would

need genuinely different query mechanisms — two pure SQL, two needing MongoDB's telemetry-level detail — so the final set of questions itself demonstrates the value of having integrated two different sources, rather than being four arbitrary trivia questions.

Q86. Looking back, what part of the project took the most time, and was that expected? *[Personalize honestly: for many people in a project like this, the data cleaning and integration diagnosis — discovering and fixing the MongoDB collection fragmentation, and tracking down the "None"-string bug in RQ3 — takes disproportionately long relative to how simple the eventual fixes turn out to be, which is a very common and realistic experience in real data engineering work. Adjust to reflect what actually took you the longest.]*

Q87. If another student picked the exact same two data sources, how would you expect their project to differ from yours? The acquisition and storage choices would likely converge, since the sources' structure genuinely calls for a relational/document split, but the specific research questions, the exact key-mapping edge cases discovered, and the depth of the data-quality assessment could all differ significantly — a project like this is less about arriving at "the one correct answer" and more about how rigorously and transparently you handle the inevitable messiness of real-world data.